

自然语言处理

王刚

统计与数据科学学院

2026 年春季学期

目录

第十章 智能问答

10.1 智能问答概述

10.2 阅读理解

10.3 表格问答

10.4 社区问答

10.5 开放领域问答

第十章 智能问答

引言

- ▶ 智能问答 (Question Answering, QA) 旨在自动回答用户以自然语言方式提出的各类问题。
- ▶ 自 1950 年图灵测试 (Turing Test) 提出以来，以自然语言进行人机交互就成为人们不断追求和奋斗的目标。这其中如何自动回答用户的各类问题，也成为了自然语言处理领域的研究热点和难点。
- ▶ 受到当前技术水平的限制，根据候选答案的来源不同以及问题种类的不同，问答系统所采用的技术手段也不尽相同。
- ▶ 近年来，随着深度学习方法不断进步，特别是超大规模预训练模型的发展，智能问答研究成果不断涌现，各类型问题的回答效果不断提高。智能问答也逐渐成为了对话助手、智能客服、搜索引擎等系统中必不可少的组成部分。

10.1 智能问答概述

智能问答概述

- ▶ **智能问答**的目标是针对用户输入的自然语言方式描述的问题自动给出答案。

20 世纪 60 年代开始自然语言处理领域就开始了相关研究，**BASEBALL**系统试图通过解析用户自然语言提出的关于美国棒球联赛的问题，通过结构化数据产生答案。

1999 年**TREC (Text Retrieval Conference)** 举办了第一届开放领域问答评测任务 TREC-8，推动了智能问答的快速发展。此后，随着搜索引擎将各种智能问答算法应用于处理用户输入的自然语言查询，使得智能问答相关算法加速发展。



智能问答发展历程

- ▶ 20 世纪 60 年代开始自然语言处理领域就开始从数据库的自然语言接口（Natural Language Interface for Database）角度开启了智能问答的研究。针对特定领域的结构化数据，通过分析用户的自然语言查询输入，并将其转换为结构化数据库查询，从而得到相应的答案。
- ▶ LUNAR 系统针对月球地质学家访问 NASA 数据库需求，构建了自然语言查询接口，可以回答如下问题：
 - (1) Which samples are breccias?
 - (2) What is the average analysis of Ir in rock S10055?
 - (3) How much Titanium does S10017 contain?

智能问答发展历程

- ▶ 20 世纪 70 年代到 80 年代，绝大部分智能问答研究仍然集中于受限领域。

SHRDLU 系统能够接受自然语言的指令，指挥虚拟积木世界中的机器人移动玩具积木块。

Chat-80 系统将英语转换为 Prolog 语言，根据知识库回答关于国家、城市、河流等世界地理知识问题。

UNIX Consultant 系统可以支持用户使用自然语言，完成对 UNIX 操作系统中特定任务的操作。这期间也有一些工作从自然语言理解的角度，试图理解文本内容，从而直接回答用户问题。

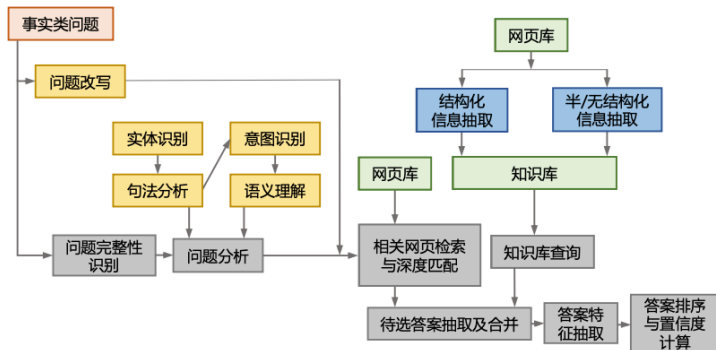
Lehnert 发布的 QUALM 系统提出了阅读理解概念，针对不同的问题类型使用不同的策略从文章中寻找答案。

智能问答发展历程

- ▶ 20 世纪 90 年代开始，随着互联网的发展和统计机器学习技术的不断进步，更实用的开放领域问答系统开始兴起。
- ▶ 1993 年由麻省理工学院 Boris Katz 教授带领 InfoLab 实验室开发的第一个基于互联网的智能问答系统 START 上线，开启了以整个互联网为知识库的开放领域问答时代。
- ▶ 1999 年开放领域问答评测任务 Trec-8 的提出，更进一步推动了开放领域问答的研究。TREC-8 评测要求根据给定的数据集合，回答事实性的短答案问题，要求系统返回答案和对应的文档编号。评测的问题相对比较简单，例如：“How many calories are there in a Big Mac?”。
- ▶ IBM 构建的 Watson 系统参加了多次 TREC 智能问答评测，并于 2011 年参加了美国电视问答比赛界面 Jeopardy! 并战胜了人类冠军。
- ▶ 2017 年搜狗问答机器人汪仔在江苏卫视问答节目《一站到底》中也战胜了人类选手取得最终胜利。

智能问答发展历程

► 搜狗问答机器人汪仔问答系统结构图



智能问答发展历程

- ▶ 随着深度学习方法在自然语言处理领域取得突破，阅读理解、社区问答、表格问答、知识图谱问答等研究领域在 2012 年以后也都取得了长足的进步。
- ▶ 特别是 2016 年斯坦福大学发布的阅读理解数据集 SQuAD，极大的推动了阅读理解的研究进展。SQuAD 包含了 10 万个高质量的问题和对应的答案，为深度学习方法提供了充足的训练语料，同时也初步验证了在训练数据充足的情况下，深度学习算法甚至可以取得超过人类的性能。
- ▶ 2017 年哈尔滨工业大学讯飞联合实验室也发布了中文机器阅读理解评测 (CMRC)。2018 年百度发布了中文阅读理解数据集 DuReader，包含 20 万个问题、100 万个文档和超过 42 万个人工给出的答案。此后大量智能问答相关评测集合相继发布，包括：多步推理阅读理解评测 HotpotQA、对话问答数据集 CoQA、复杂序列问答 SequenceQA、自由形式表格问答 FeTaQA 等。

智能问答主要类型

- 受到现有机器学习方法和自然语言处理算法能力的限制，目前还没有通用方法可以回答所有类型的问题，根据问题类型以及答案来源的不同，需要采用不同的方法进行解决。可以将问题大致分为七类：事实类 (Factoid)、是非类 (Yes/No)、定义类 (Definition)、列表类 (List)、比较类 (Comparison)、意见类 (Opinion) 以及指导类 (How-to)。

问题类型	问题	答案
事实类	复旦大学成立于那年？	1905 年
是非类	复旦大学在上海吗？	是
定义类	什么是自然语言处理？	实现人与计算机之间用自然语言进行有效通信的各种理论和方法
列表类	复旦大学有哪几个校区？	邯郸、枫林、张江、江湾
比较类	上海和北京哪个人口多？	根据第七次全国人口普查北京市人口数为 2189.31 万，上海市人口数为 2487.09 万，上海人口比北京多
意见类	你觉得上海哪些最值得去的地方？	上海这座被称为“魔都”的城市有很多知名的经典,我觉得最值得去的地方包括外滩、东方明珠、城隍庙、豫园、田子坊等
指导类	如何从虹桥机场到浦东机场？	可以有以下三种方式：1) 机场大巴直达：机场一线；2) 乘坐地铁 2 号线直达；3) 乘坐地铁 2 号线到龙阳路站换乘磁悬浮列车

智能问答主要类型

- ▶ **阅读理解 (Machine Reading Comprehension, MRC)**，也称机器阅读理解，是指根据给定的一篇或多篇文本内容回答给定的问题。按照答案类型的不同，还可以进一步细分为完型填空、多项选择、片段抽取和自由作答四种形式。
- ▶ **表格问答 (Table based Question Answering, TBQA)** 是指根据给定的表格数据生成问题答案。表格通常由 M 行 N 列的数据组成，第一行中的 N 个单元格是表头信息。
- ▶ **社区问答 (Community Question Answering, CQA)** 是指根据社区问答等来源获得的 < 问题, 答案 > 对进行问题回答。社区问答根据用户输入的问题，从已有的问答对中寻找语义最相关的问答，并将所对应的答案返回给用户。
- ▶ **知识图谱问答 (Knowledge based Question Answering, KBQA)** 是指根据给定知识图谱生成问题的答案。根据用户的问题，在图谱中根据实体和关系并利用推理机制可以进行问题回答。
- ▶ **开放领域问答 (Open-domain Question Answering, ODQA)** 是通过大规模文档集合回答不限定领域的事实性问题。通常情况下开放领域问答由答案段落检索和答案抽取两个部分组成。

10.2 阅读理解

阅读理解

机器阅读理解目标：根据给定的一篇或多篇文本内容回答给定的问题。

- ▶ 这个任务对于算法的自然语言理解能力和推理能力是很大的考验。虽然阅读理解任务研究在 1999 年就已经开始，但是受到基于特征的机器学习算法能力的限制，以及缺乏大规模的标注数据，阅读理解的研究进展相对较慢。
- ▶ 2016 年斯坦福大学推出了 SQuAD 语料集，大幅度推动了人们对阅读理解研究的关注，大量深度神经网络的模型不断提出。2018 年 1 月，微软亚洲研究院提出的 R-Net 算法率先在 SQuAD 的精准匹配指标上首次超越人类。2018 年基于预训练语言模型 BERT 的阅读理解算法在该任务上取得了大幅度进展。

阅读理解

- ▶ 目前绝大多数的机器阅读理解算法都采用有监督方法，**将阅读理解任务转换为分类问题**，因为不同的答案形式需要所采用的方法有所不同，由此可以将阅读理问题分为以下四大类：
 - (1) 完型填空：对于一段文本中的某个句子，其中缺少某个单词或短语，需要算法根据其他的文本内容预所缺失部分。
 - (2) 多项选择：给定文本内容，问题及对应的若干选项，需从选项中选择出一个或多个正确的选项构成答案。
 - (3) 片段抽取：给定文本内容和问题，从文本中抽取单词、短语、句子或者段落作为答案。
 - (4) 自由作答：给定文本内容和问题，生成一段可以回答问题的文字。答案可能并没有出现在给定文本中。

基于特征的阅读理解算法—基于规则的 Quarc

- ▶ Quarc (QuesTion Answering for Reading Comprehension) 是一个基于规则的阅读理解系统，面向片段抽取式的阅读理解类型，并且抽取的答案限定于句子级别，试图在给定的文章中找到最合适的句子作为问题的答案。
- ▶ Quarc 使用词汇和语义作为基础规则条件，通过启发式的规则来寻找最合适的句子作为答案。
- ▶ 由于不同类型的问题所使用的规则有比较大的差别，Quarc 系统将问题分为：WHO、WHAT、WHEN、WHERE、WHY 等 5 个类型，并针对不同类型的问题构建了不同的规则集。
- ▶ Quarc 算法首先使用浅层句法解析器 Sundance 进行形态分析、词性标注、语义类标注和实体识别，解析问题和文章中的所有句子。为了避免直接使用单词造成的规则的泛化能力不足的问题，Quarc 算法在对单词进行比对时会使用词根，用于消除语法形式的影响。

基于特征的阅读理解算法—基于规则的 Quarc

- ▶ 此外还定义了语义类（例如 HUMAN、LOCATION 等），并使用规则方式识别单词的语义类。主要的语义类别包含以下几项：

HUMAN：包含 2608 个单词，包含常见的名，姓氏和例如“博士”和“女士”等头衔，并且包含 600 个从 WordNet 中抽取的职业词。

LOCATION：包含 344 个单词，包含 204 个国家名和 50 个美国的州名。

MONTH：包含一年中的 12 个月。

TIME：共包含 667 个词语，其中 600 个是 1400-1999 的年份，其他是一些通用的时间表达。

基于特征的阅读理解算法—基于规则的 Quarc

- 在此基础上，根据问题的类型选择不同的规则集合，将这些规则应用于文章中包括标题在内的所有句子。每条规则都会给予句子一定的分数，分数的高低取决于该规则对于它能找到答案的确定程度。
一条规则可以分配四种类型的分数：线索 (clue)(+3)，好线索 (good_clue)(+4)，确信 (confident)(+6)，完全契合 (slam_dunk)(+20)。当所有规则都执行完毕后，得到分数最高的句子将被选作答案。以 WHERE 类问题的规则集合为例，Quarc 算法中 WHERE 类部分规则如下：

[1] Score(S) += WordMatch(Q,S)

[2] If contains(S,LocationPrep) Then Score(S) += good_clue

[3] If contains(S, LOCATION) Then Score(S) += confident

规则 [1] 中 WordMatch 函数是根据同时出现在句子和问题中的单词计算的得分。当两个单词拥有相同的词根时会认为这两个单词匹配。由于动词对于识别问题和句子的相关性非常重要，所以动词匹配的权重比非动词更高：每个匹配的动词被授予 6 分，其他匹配词仅有 3 分。

规则 [2] 中 LocationPrep 表示地点介词（包括：in、at、near 等），该规则试图寻找句子 S 中包含地点介词，对于包含地点介词的句子增加“好线索”类对应的分数。

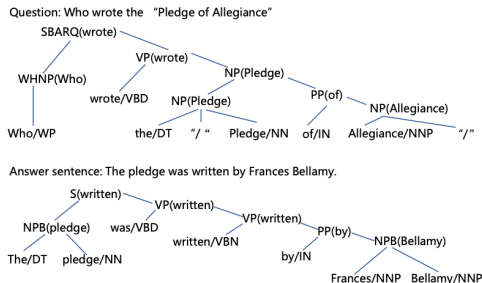
规则 [3] 试图寻找包含 LOCATION 语义类的句子，相应的句子增加“确信”类所对应的分数。

基于特征的阅读理解算法—基于规则的 Quarc

- ▶ 当所有规则应用于文章中的每个句子，并分配了分数之后，拥有最高分数的句子就被当成是最佳答案。如果有相同分数的句子，对于原因类的问题将会选择出现更晚的句子，而其他问题都会选择出现最早的句子。如果所有句子都没有得到分数，时间和地点类的问题将返回根据日期线规则集得到的句子，原因类问题将选择文章中的最后一个句子，其它类型都将返回文章的第一个句子。
- ▶ Quarc 算法是典型的基于规则的自然语言处理方法，在对篇章进行词法、句法和语义分析的基础上，构建人工规则。在阅读理解这种复杂的自然语言处理任务上，基于规则的方法效果并不十分理想。

基于特征的阅读理解算法-基于最大熵的阅读理解算法

- ▶ RCME 算法在词袋模型的基础上，引入了两类深层特征：**词依赖特征**和**语法关系特征**，并采用最大熵算法将两类特征加以利用。
- ▶ 为了更好的提取深层特征，RCME 算法需要对输入文本中的每个句子进行语法分析，得到整个句子的语法结构和每个单词之间的依赖关系和词性。
- ▶ RCME 模型将句子中每个单词的词性抽取出来作为词依赖特征，将词之间的语法关系作为语法关系特征，利用这两个深层特征去判断给定文本中的哪一句话最适合作为问题的答案。



基于特征的阅读理解算法-基于最大熵的阅读理解算法

- ▶ 例如：对于下面的问题 Q 和句子 C

Q: 谁写了《红楼梦》?

C: 《红楼梦》是曹雪芹写的

其中部分词和词性的集合如下：

Q = 写/动词, 红楼梦/名词

C = 写/动词, 红楼梦/名词, 曹雪芹/名词

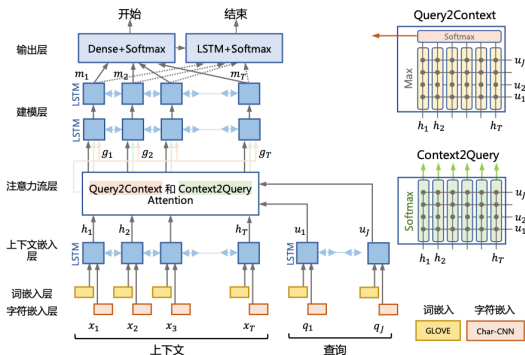
- ▶ RCME 首先定义了特征函数 $f_j(Q, C)$, 对于问题 Q 中的每一个特征 f_j , 当句子 C 中包含该特征时, $f_j(Q, C) = 1$, 反之, $f_j(Q, C) = 0$ 。在上面的例子中, $f_{\text{动词}}(Q, C) = 1$, $f_{\text{名词}}(Q, C) = 1$ 。RCME 利用最大熵的框架利用这些特征挑选最适合回答问题的句子: 对于给定的文本 $S = \{C_1, \dots, C_n\}$ 和给定问题 Q, 模型需要从中挑选最适合的回答:

$$A = \arg \max_{C_i \in S} P(C_i | Q)$$

$$P(C_i | Q) = \frac{\exp \left(\sum_j \lambda_j f_j(Q, C_i) \right)}{\sum_C \exp \left(\sum_j \lambda_j f_j(Q, C) \right)}$$

基于深度神经网络的阅读理解算法-双向注意力流网络

- ▶ BiDAF是基于双向注意力流 (Bi-directional Attention Flow) 的阅读理解算法。针对的是 SQuAD 评测集合, 给定文章和问题, 算法需要返回文章中的一个片段作为答案, 答案片段通常是短语。
- ▶ BiDAF 将该任务建模为将篇章和问题作为输入, 预测开始位置和结束位置的问题。



基于深度神经网络的阅读理解算法-双向注意力流网络

- ▶ **字符向量层 (Character Embed Layer)** 作用是利用卷积神经网络对文本和问题中每个单词的字符进行建模，计算得到每个单词相应的字符表示。
- ▶ **词向量层 (Word Embed Layer)** 将问题和文章中的每个单词生成对应的词向量表示，BiDAF 使用了 Glove 生成文本和问题的词向量，并构成文章表示矩阵 $X \in \mathbb{R}^{d \times T}$ 和问题表示矩阵 $Q \in \mathbb{R}^{d \times J}$ 。
- ▶ **上下文向量层 (Contextual Embed Layer)** 使用 Highway Networks 对前两层中生成的字符向量和词向量进行融合，并将融合后的结果输入给一个 BiLSTM 网络，分别计算得到问题和文章的隐状态序列是 $U = [u_1, u_2, \dots, u_J] \in \mathbb{R}^{2d \times J}$ 和 $H = [h_1, h_2, \dots, h_T] \in \mathbb{R}^{2d \times T}$ 。由于这里使用了 BiLSTM，因此 u_i 和 h_j 都是由前向 LSTM 和后向 LSTM 的 d 维隐变量连接而成的 $2d$ 维列向量。此外还需要注意的是，本层的计算过程中并没有对问题和文章进行信息交互。

基于深度神经网络的阅读理解算法-双向注意力流网络

- ▶ **注意力流层 (Attention Flow layer)**对文章和问题进行信息交互。首先根据文章表示矩阵 H 和问题表示矩阵 U 计算相似度矩阵 $S \in \mathbb{R}^{T \times J}$ ，其中 S_{ij} 表示输入文本中第 i 个单词和给定问题中第 j 个单词之间的相似度：

$$S_{ij} = \alpha(h_i, u_j), \quad \alpha = W_S^T[h_i; u_j; h_i \circ u_j]$$

$W_S \in \mathbb{R}^{6d}$ 是可训练参数， $h_i \circ u_j$ 是元素积， $[:]$ 表示向量按行连接。

- ▶ **建模层 (Modeling Layer)**将之前计算出的 G 输入给建模层，输出是建模层获取到的文章向量和问题向量的信息交互。模型采用 BiLSTM 网络，网络输出矩阵 M 用来预测答案。
- ▶ **输出层 (Output Layer)**利用 G 预测答案的起始位置 p^1 和答案的结束位置 p^2 ：

$$p^1 = \text{Softmax}(W_{p^1}^T[G, M])$$

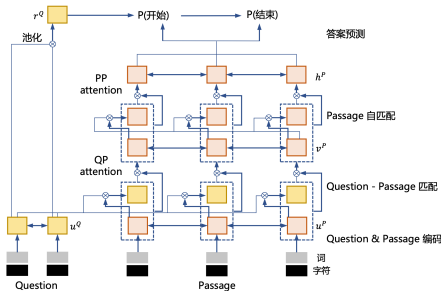
$$p^2 = \text{Softmax}(W_{p^2}^T[G, M^2])$$

W_{p^1} 和 W_{p^2} 可训练参数， M^2 是基于 M 再采用一次 BiLSTM 网络生成的另外一个隐向量序列。

基于深度神经网络的阅读理解算法-门控自匹配注意力

- **R-Net**是基于门控自匹配注意力机制的阅读理解模型。R-Net的创新主要有两点：

- (1) 在 Match-LSTM 网络的基础上进行改进，引入了门控的机制来学习输入文本的哪一部分与问题的相关性最大；
- (2) 自匹配机制，本质是自注意力，充分学习了文本间词和词之间的关系，而不是只关注于文本和问题之间的关系，R-Net 之前的工作，只是利用一个双向 LSTM 对输入文本进行特征提取，而没有进行更深的挖掘。



基于深度神经网络的阅读理解算法-门控自匹配注意力

- ▶ R-Net 采用两个不同的 BiGRU 网络分别作为问题编码器和文本编码器，对问题 $Q = [w_1^Q, \dots, W_m^Q]$ 和文本 $P = [w_1^P, \dots, w_n^P]$ 中每个单词所对应的词向量和字符向量表示进行编码，生成对应的隐状态序列：

$$u_t^Q = \text{BiGRU}(u_{t-1}^Q, [e_t^Q, c_t^Q])$$

$$u_t^P = \text{BiGRU}(u_{t-1}^P, [e_t^P, c_t^P])$$

u_t^Q, u_t^P 分别表示问题和文本中第 t 个单词所对应的隐向量表示， e_t^Q, e_t^P 分别表示问题和文本中第 t 个单词所对应的词向量， c_t^Q, c_t^P 代表字符向量。

- ▶ 为了充分挖掘输入文本中与给定问题相关程度最大的部分，R-Net 对输入文本的特征提取上进行了改进，不仅仅是对整个输入文本进行处理，而是更细粒度的关注文本中的不同部分与问题的关系。

基于深度神经网络的阅读理解算法-门控自匹配注意力

- 为此，R-Net 模型采用了以门控注意力机制为基础的循环网络将给定问题与输入文本对应的向量表示相融合。首先，将字符向量和单词向量连接在一起，输入 RNN 网络，对文章总体进行语义建模，并通过注意力机制对上下文向量进行加强。得到上下文向量 c ：

$$v_t^P = \text{RNN}[v_{t-1}^P, [u_t^P, c_t]]$$

$$c_t = \sum_{i=1}^m a_i^t u_i^Q$$

$$a_i^t = \text{Softmax}(s^t)$$

$$s_j^t = v^T \tanh(W_u^Q u_j^Q + W_u^P u_t^P + W_v^P v_{t-1}^P)$$

然后，通过门控注意力，将问题信息和文本信息进行充分融合：

$$g_t = \text{Sigmoid}(W_g[u_t^P, c_t])$$

$$u_t^P = g_t \cdot u_t^P$$

$$c_t = g_t \cdot c_t$$

基于深度神经网络的阅读理解算法-门控自匹配注意力

- ▶ 模型采用自匹配注意机制生成文本向量表示序列

$H = [h_1^P, \dots, h_n^P]$ 对于阅读理解任务来说, 输入文本中每个单词的向量表示不仅仅与问题中的单词有关, 还应该与该单词的上下文有关, 自匹配注意力机制就是要将这两种信息融合起来。

$$h_t^P = \text{BiRNN}(h_{t-1}^P, [v_t^P, \tilde{c}_t])$$

$$\tilde{c}_t = \sum_{i=1}^n a_i^t v_i^P$$

$$a_i^t = \frac{\exp(\tilde{s}_i^t)}{\sum_{j=1}^n \exp(\tilde{s}_j^t)}$$

$$\tilde{s}_j^t = \tilde{v}^t \tanh(\tilde{W}_v^P v_j^P + \tilde{W}_v^P v_t^P)$$

基于深度神经网络的阅读理解算法-门控自匹配注意力

- 最后 R-Net 利用 Pointer Network 对答案的起始位置 p^1 和结束位置 p^2 进行预测:

$$p^t = \arg \max(a_1^t, \dots, a_n^t)$$

$$a_i^t = \frac{\exp(s_i^t)}{\sum_{j=1}^n \exp(s_j^t)}$$

$$s_j^t = w^T \tanh(W_h^p h_j^p + W_h^a h_{t-1}^a)$$

$$h_t^a = \text{RNN}(h_{t-1}^a, c_t)$$

$$c_t = \sum_{i=1}^n a_i^t h_i^p$$

基于深度神经网络的阅读理解算法-片段掩盖的预训练

- ▶ **SpanBERT**通过对 BERT 进行扩展，可以更有效的表示和预测片段相关的任务，如抽取式问答。
- ▶ 原始 BERT 算法在训练时会随机选取句中的子词进行遮盖，但这种训练方式会让一些原本就连续出现且相关性很强的词组短语，在训练的时候被割裂开。
- ▶ SpanBERT 打破了这种范式，对于给定的文本 $X = x_1 \dots x_n$ ，通过迭代的方式从中选择需要遮盖的片段。在每一次迭代中，SpanBERT 通过几何分布对遮盖长度 l 进行随机采样， l 代表要遮盖的单词的数量。紧接着在一个均匀分布中采样出一个遮盖起点 s ，对 $\{x_s, \dots, x_{s+l}\}$ 进行遮盖，在损失函数方面，这一部分与原本的 BERT 保持一致，采用 MLM 的损失函数进行训练。

基于深度神经网络的阅读理解算法-片段掩盖的预训练

- SpanBERT 引入了一个新的训练目标, Span Boundary Objective (SBO)。SpanBERT 希望训练好的语言模型可以让每个片段的边缘单词的表示尽可能的包含片段内部内容的所有语义, 所以 SBO 的含义是要求在片段边界处的单词表示尽可能可以对遮盖片段内部的单词进行预测。具体来说, 对于给定文本 X , 对于某个被选中的遮盖片段 $\{x_s, \dots, x_{s+l-1}\}$, SpanBERT 希望通过 $\{x_{s-1}, x_{s+l}\}$ 以及位置向量 p_{i-s+1} 对被遮盖的单词 x_i 进行预测:

$$y_i = f(h_{s-1}; h_{x_{s+l}}; p_{i-s+1})$$

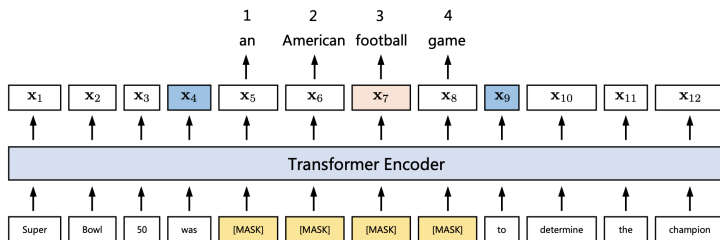
最终 SpanBERT 的损失函数为:

$$\mathcal{L}(x_i) = \mathcal{L}_{\text{MLM}}(x_i) + \mathcal{L}_{\text{SBO}}(x_i) = -\log P(x_i|X) - \log P(x_i|x_{s-1}, x_{s+l}, p_i)$$

基于深度神经网络的阅读理解算法-片段掩盖的预训练

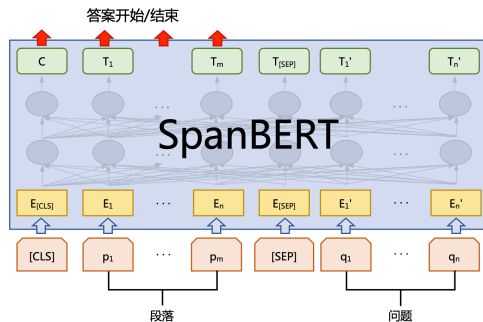
- ▶ **片段掩盖的样例**: 该例子中 an American football game 全部使用 [MASK] 替代， x_4 和 x_9 以及位置编码用于预测掩盖区域内的单词。在本例中 football 是掩盖片段中的第 3 个单词，因此采用 p_3 的位置编码。football 所对应的损失函数为：

$$\mathcal{L}(\text{football}) = -\log P(\text{football} | x_7) - \log P(\text{football} | x_4, x_9, p_3)$$



基于深度神经网络的阅读理解算法-片段掩盖的预训练

- 在预训练完成的基础上，采用与 BERT 相同的建模方式，将段落 $P = (p_1, p_2, \dots, p_m)$ 和问题 $Q = (q_1, q_2, \dots, q_n)$ 编码为一个序列 $X = [\text{CLS}]p_1p_2\dots p_n[\text{SEP}]q_1q_2\dots q_m$ 作为输入。在段落单词所对应的位置，训练两个线性分类器，分别建模该单词是否为答案片段的开始还是结束位置。为了应对 SQuAD 2.0 中存在不能回答的问题的情况，将答案片段的开头和结尾都标记于 $[\text{CLS}]$ 所对应的位置。



阅读理解语料库

数据集	语言	任务类型	数据来源	问题数量	文档数量
CNN / Daily Mail	英语	完形填空	自动生成	140 万	30 万
Children's Book	英语	完形填空	自动生成	68.8 万	68.8 万
HLF-RC	中文	完形填空	自动生成	10 万	2.8 万
MCTest	英语	多项选择	人工编写	2640	660
RACE	英语	多项选择	英文测试题	87 万	5 万
SQuAD	英语	片段抽取	人工编写	10 万	536
TriviaQA	英语	片段抽取	用户问答界面	4 万	66 万
NewsQA	英语	片段抽取	人工编写	10 万	1 万
MS-MARCO	英语	自由问答	用户日志	10 万	20 万
DuReader	中文	自由问答	用户日志	20 万	100 万

阅读理解语料库

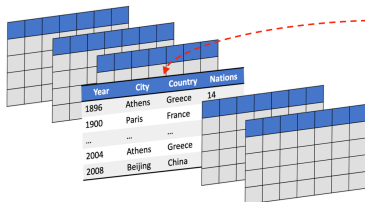
- ▶ **SQuAD**是斯坦福大学于 2016 年推出的阅读理解数据集，给定一篇文章和相应问题，需要模型给出问题的答案，SQuAD 是片段抽取类任务，答案为文章中的片段。数据集的所有文章都选自维基百科，一共有 107,785 个问题，以及配套的 536 篇文章。SQuAD2.0 已发布，其中包含无法回答的问题。SQuAD 的主要缺点是数据集中，问题的答案一般是很简短的一个词组。
- ▶ **MS MARCO**又称人类生成的机器阅读理解数据集，由 Microsoft AI & Research 设计和开发。数据集中的所有问题都是从真实的匿名用户查询中获得的，研究者们为这些问题编写了答案。问题配套的文章是通过 Bing 搜索引擎从真实文档中提取可以获取答案的上下文段落。该数据集有 1010916 个数据。
- ▶ **TrivialQA**数据集是一个难度较高的阅读理解数据集，其中问题较为复杂，并且问题和文章中相应的支持句之间有较大的差距，例如一些句法或者词法的变化，并且相比其他阅读理解数据集，TrivialQA 中包含更多的需要多跳推理能力的问题。

10.3 表格问答

表格问答

- 表格是一种常见的数据存储形式，通常由表头，表格单元和表格标题构成：表格标题概括了表格包含的主要内容，表头一般表示某行或者某列表格单元的内容和类型。除此之外，表格中每个元素都是一个表格单元。

下图给出了表格问答样例。对于问题“哪个城市举办了2008年奥运会？”，通过表格选择和单元格抽取，最终返回“北京”作为答案。



Year	City	Country	Nations
1896	Athens	Greece	14
1900	Paris	France	24
...	...	...	...
2004	Athens	Greece	201
2008	Beijing	China	204

问题：哪个城市举办了2008年奥运会？



表格选择

Year	City	Country	Nations
1896	Athens	Greece	14
1900	Paris	France	24
...	...	...	...
2004	Athens	Greece	201
2008	Beijing	China	204

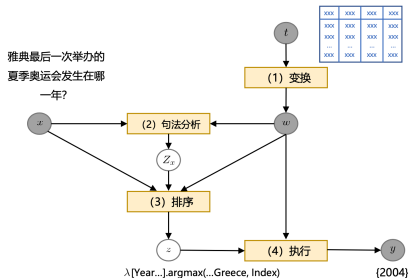


单元格抽取

答案：北京

表格问答 – 基于特征的表格问答方法

- ▶ 对于解决表格问答问题，一个基本的想法是将输入的表格视为一个知识库，但这个过程本身包含着很多挑战，因为知识库包含了太多的关系，这令表格数据充满噪音。
- ▶ CSP 算法是基于特征的表格问答方法，采用逻辑形式驱动。对于一个给定的表格 t 和一个关于表格的问题 q ，模型需要根据表格内容输出答案 y 来回答 q 。对于问题 q 唯一的限制是这个问题需要仅能够通过表格内容回答，问题可以是任何形式。CSP 算法的整体流程如图所示。



表格问答 – 基于特征的表格问答方法

- 为了解决表格知识库的噪音问题，CSP 模型首先将表格 t 转化成知识图谱的形式。具体来说，将表格的行视为行节点，表格单元中的字符串被视为实体节点，表格的列被转化成连接行节点和实体节点的有向边，而列表头则成为了这些边的标签。
- 得到知识图谱 w 之后，CSP 模型将问题 q 解析成一些候选的逻辑形式 Z_x 。CSP 模型提出了表单检索的方法，对于给定的问题 $q = q_1, \dots, q_n$ ，模型通过两类规则对问题进行解析：

$$(q, i, j)[s] \rightarrow (c, i, j)[f(s)]$$

$$(c_1, i, k)[z_1] + (c_2, k + 1, j)[z_2] \rightarrow (c, i, j)[f(z_1, z_2)]$$

第一个规则是一类词法匹配规则，可以将问题片段 $q_i \dots q_j$ 转化为逻辑形式，其中 c 表示类别， s 表示问题片段的文本。例如， s 为“纽约”， c 为“实体”时，根据表格查询内容 $f(s)$ 为“纽约市”。

第二个规则将两个相邻片段的逻辑形式 z_1 和 z_2 合并成一个新的逻辑形式 $f(z_1, z_2)$ 。

表格问答 – 基于特征的表格问答方法

- ▶ CSP 模型将以上特征 $g(x, w, z)$ 输送给逻辑线性模型去捕捉问题 q 和候选 z 之间的关系。
- ▶ CSP 模型利用解析生成出逻辑形式 Z_x ，每一个逻辑形式 z_i 都是一个从知识图中抽取出的图问题，利用提取出的特征向量 $g(x, w, z)$ ，CSP 模型定义了对于每个逻辑线性分布：

$$p(z|x, w) = \exp(\theta^T g(x, w, z))$$

- ▶ **模型的训练方式**：对于给定的训练样本 $D = \{(x_i, t_i, y_i)\}_{i=1}^N$ ，CSP 模型在最大似然估计的基础上增加正则项作为目标函数：

$$L = \frac{1}{N} \sum_{i=1}^N \log p(y_i | q_i, w_i) - L_1$$

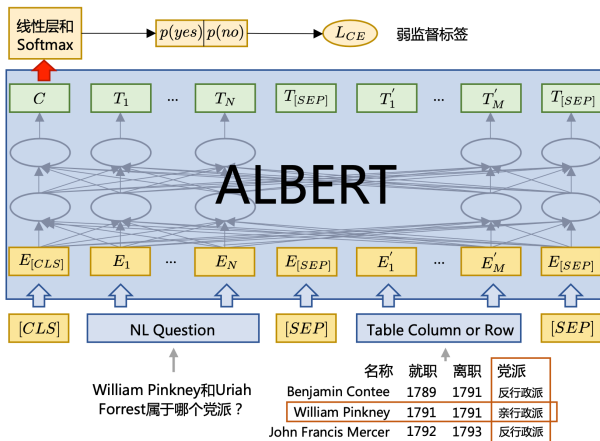
其中 L_1 是模型的 L_1 正则， w_i 是根据 t_i 动态生成的：

$$p(y|x, w) = \sum_{z \in Z_x} p(z|x, w)$$

- ▶ 在训练完毕之后，模型挑选分数最高的逻辑形式 z 作为答案。

表格问答 – 基于深度学习的表格问答模型

- **CLTR**是基于预训练的端到端表格问答模型，该方法首先从大量表格中筛选得到少量相关表格，再对这些表格进行重新排序，最终回答用户提出的自然语言问题。其模型神经网络结构如图所示。



表格问答 – 基于深度学习的表格问答模型

- ▶ CLTR 模型首先利用 BM25¹ 算法筛选出一个高相关表格池，接下来采用行列交互模型对表格池中表格中的行列生成包含答案的概率。行列交互模型将表格问答分为两个基本操作：行选择和列选择，将行列的预测分布进行合并，就能得到答案单元格的目标概率。

CLTR 采用文本分配的方式预测答案所在的行列，具体来说就是将每行每列分别与问题 q 相连输入 BERT 模型中，通过二分类器得到概率分数：

$$H_{r_i} = \text{BERT}(q, r_i), \quad H_{c_j} = \text{BERT}(q, c_j)$$

$$P_{r_i} = \text{Binary Classifier}(H_{r_i}), \quad P_{c_j} = \text{Binary Classifier}(H_{c_j})$$

CLTR 模型利用最大单元分数对表格池里的候选表格进行重排序，当重排序做完之后，会挑选表格池中得分前 k 的表格返回给用户。最后的答案由行列交互模型所计算的分数，寻找分数最高的交叉行列来得到目标表格单元。

¹BM25 是信息索引领域最主流的计算 query 与文档相似度得分的算法。

表格问答语料库

- ▶ WikiSQL 和 Spider 数据集合采用用户真实问句，并人工转换为数据库 SQL 查询语句的方式。WikiTableQuestions 则是来源于维基百科的半结构化表格。

数据集	数据来源	表格数量	问句数量
WikiSQL	用户真实问句转换	24241	80645
Spider	用户真实问句转换	206	10181
WikiTableQuestions	维基百科	2108	22033

10.4 社区问答

社区问答

- ▶ 随着 Web 2.0 的快速发展，社区问答网站自 2008 年以来蓬勃发展，出现了包括 Quora、知乎等在内的众多社区问答网站。用户可以通过这些网站对自己需要解决的问题进行提问，其他用户对这些问题进行回答、关注、评论等。
- ▶ 企业内部也大量问答知识，例如人工客服领域，很多高频问题都有相应的知识库。基于这些问答数据，研究人员们提出了社区问答任务，基于收集的 < 问题，答案 > 对数据，针对用户自然语言问题寻找最合适的答案返回。



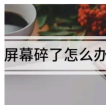
搜狗搜索

手机屏幕碎了怎么办？



搜狗搜索

手机屏幕碎了怎么办修复 立知



屏幕碎了怎么办

手机屏幕碎了要分清楚是外屏碎了，还是内屏...

1.外屏碎了需要换新屏幕， 首先将手机关机后...

2.将手机和盖背与上部分分开。

3.分开之后将新屏幕装在盖背上， 再将螺丝上...

更多 >

懂视网

👍 3

📝 反馈

社区问答

► 社区问答方法可以进一步细分为两个子任务：

1) 问题-问题匹配：目标是计算输入问题和问答库中某个已有问题 Q_i 之间的相似度，如果两者之间相似度较高，显而易见的， Q_i 的答案 A_i 就有很大概率是问题 Q 的答案。

2) 问题-答案匹配：目标计算输入问题 Q 与问答库中某个答案 A_i 的相关性。同样的，如果两者之间相关度很高，那么 A_i 就有很大概率是问题 Q 的答案。

基于特征的语义匹配

- 传统的基于特征的语义匹配方法普遍采用一些基于词袋的统计文本表示，通过这些表示去计算两段文本的语义相似度，进而判断两段文本的语义是否匹配。其中，n 元短语匹配度是判断语义相似度的一个重要衡量指标，如果两个问题共同包含的语义片段越多，自然具有更大的可能表达相同的含义。n 元短语匹配度有许多不同的计算方式，最简单的做法可以计算两个问题中重复单词的数量：

$$\text{Sim}(Q_1, Q_2) = \frac{\sum_{w \in Q_1} \text{num}(w, Q_2)}{|Q_1|}$$

$\text{num}(w, Q_2)$ 表示单词 w 在 Q_2 中出现的次数， $|Q_1|$ 表示 Q_1 的长度。

- 类似地，可以将简单的单词匹配扩展到短语匹配，由于短语包含的信息量更多，表达的语义也更加独立，所以短语级匹配特征能更好的表达两个问题之间的语义相似度。常见的短语级匹配特征有 BLEU、编辑距离和最长公共子序列方法都可以将它们用来当作特征来完成问题匹配的任务。

基于特征的语义匹配

- ▶ 这些语义匹配特征都存在两个 **缺点**：首先，没有处理停用词，这类词汇不具备实际意义，对语义的贡献程度小。第二，没有处理同义词，由于自然语言的表达非常丰富，同一语义具有非常多的表达方式，这就使得处理语义匹配问题时需要能够对同义词有较好的识别能力。
- ▶ BM25 可以缓解停用词带来的问题，使用基于反转文档频率计算 Q_1 和 Q_2 的相似度：

$$f_{\text{BM25}}(Q_1, Q_2) = \sum_{i=1}^{|Q_1|} \text{IDF}(Q_1^i) \text{freq}(Q_1^i, Q_2)$$

$$\text{freq}(Q_1^i, Q_2) = \frac{\text{num}(Q_1^i, Q_2)(k_1 + 1)}{\text{num}(Q_1^i, Q_2) + k_1(1 - b + b \times \frac{|Q_2|}{\text{avglen}(Q_2)})}$$

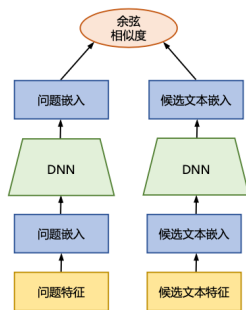
$$\text{IDF}(Q_1^i) = \log \frac{N - n(Q_1^i) + 0.5}{n(Q_1^i) + 0.5}$$

$$\text{BM25}(Q_1, Q_2) = \frac{f_{\text{BM25}}(Q_1, Q_2) + f_{\text{BM25}}(Q_2, Q_1)}{2}$$

通过 BM25 的特征值，可以通过检测句对之间的 BM25 得分是否超过阈值来判断两个问题之间是否匹配。

基于深度神经网络的问题匹配—DSSM 语义匹配

- ▶ 基于深度学习的语义匹配主要分为交互式和非交互型。非交互型的方法是将两个句子分别编码后进行匹配，交互式的方法一般是将两个句子一起进行编码，在编码过程中让两个句子进行信息交互。
- ▶ DSSM (Deep Structured Semantic Model) 是一种非交互式的基于深度网络的文本语义匹配算法，通过深度神经网络对用户问题和题目分别进行建模，将其编码成低维度的语义向量，然后计算两个向量的余弦距离，判断两个句子之间的语义相似度，由于该模型对用户问题和题目分别进行建模，在计算相似度之前两者之间没有交互，这种模型架构又被称作“双塔模型”。



基于深度神经网络的问题匹配—DSSM 语义匹配

- ▶ 令 x 表示输入向量, y 表示输出向量, 向量 y 的计算方式如下:

$$l_1 = W_1 x$$

$$l_i = f(W_i l_{i-1} + b_i), i = 2, \dots, N - 1$$

$$y = f(W_N l_{N-1} + b_N)$$

其中 l_i 表示隐藏层, 共 N 层, W_i 表示第 i 层的参数矩阵, b_i 表示偏置项, f 是输出层和隐藏层的激活函数, 采用 $\tanh$ 函数。

- ▶ 在社区问答任务中使用 Q 表示用户查询, t 表示候选文档题目, 两者之间的相关性分数计算方式如下:

$$\text{Score}(Q, t) = \cos(y_Q, y_t)$$

基于深度神经网络的问题匹配—DSSM 语义匹配

- DSSM 模型利用点击日志中用户搜索的问题和用户点击的文档作为数据进行训练。其基本假设是：如果用户在当前的搜索问题下点击了某个文档，那么该文档和问题之间一定是相关的。利用该假设通过搜索日志自动构建训练集和测试集。模型在学习过程中利用极大似然估计作为损失函数：

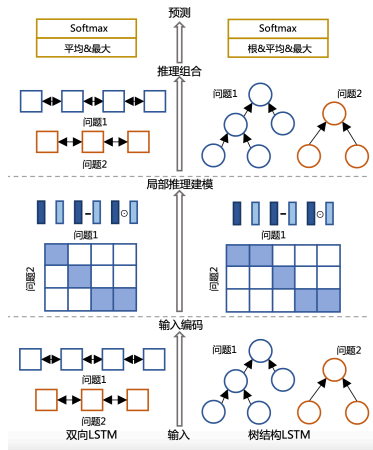
$$\mathcal{L} = -\log \prod_{Q, D^+} P(D^+|Q)$$

$$P(D^+|Q) = \frac{\exp(\text{Score}(Q, D^+))}{\sum_{d \in D} \exp(\text{Score}(Q, t))}$$

其中， D 表示候选文档集合， D^+ 为正样本。

基于深度神经网络的问题匹配—ESIM 语义匹配

- ESIM 模型基于链式 LSTMs 设计了序列推断模型，并考虑了局部推断和推断组合的问题，在模型中引入了句子间的注意力机制来实现局部推断进而实现全局推断。ESIM 模型结构主要分为三个部分：输入编码层、局部推断模块以及推断组合模块。



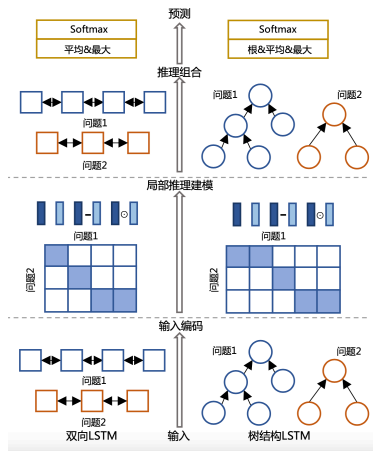
基于深度神经网络的问题匹配—ESIM 语义匹配

- 输入编码层对于输入的两个问题 Q_1 和 Q_2 ，模型首先通过词嵌入表示得到问题中每个单词的词向量，然后将其输入 BiLSTM 模型，通过 BiLSTM 表达句子的局部信息：

$$h_1^i = \text{BiLSTM}(h_1^{i-1}, h_1^{i+1}, q_1^i)$$

$$h_2^i = \text{BiLSTM}(h_2^{i-1}, h_2^{i+1}, q_2^i)$$

其中 q_1^i 和 q_2^i 分别表示 Q_1 中的第 i 个单词和 Q_2 中的第 i 个单词的嵌入表示。



基于深度神经网络的问题匹配—ESIM 语义匹配

- ▶ 局部推断模块在进行局部推断之前，首先模型要对 h_1 和 h_2 进行对齐，首先计算两者的相似度:

$$e_{ij} = h_1^T h_2$$

进行局部推断，结合 h_1 ， h_2 和 e_{ij} ，得到相似性加权后的向量:

$$\hat{h}_1^i = \sum_{j=1}^{\text{len}_2} \frac{\exp(e_{ij})}{\sum_{k=1}^{\text{len}_2} e_{ik}} h_2^j$$

$$\hat{h}_2^i = \sum_{j=1}^{\text{len}_1} \frac{\exp(e_{ij})}{\sum_{k=1}^{\text{len}_1} e_{ik}} h_1^j$$

在局部推断之后，模型会进行局部信息增强，利用差和点积放大两者的差异性:

$$m_1 = [h_1, \hat{h}_1, h_1 - \hat{h}_1, h_1 \cdot \hat{h}_1]$$

$$m_2 = [h_2, \hat{h}_2, h_2 - \hat{h}_2, h_2 \cdot \hat{h}_2]$$

基于深度神经网络的问题匹配—ESIM 语义匹配

- 推断组合模块将 m_1 和 m_2 输入 BiLSTM 提取信息:

$$v_1^i = \text{BiLSTM}(m_1^i)$$

$$v_2^i = \text{BiLSTM}(m_2^i)$$

由于两个问题的长度不同导致 m_1, m_2 矩阵维度不一致, ESIM 分别采用 MaxPooling 和 AvgPooling 的池化对两个句子进行处理:

$$v_1^{\max} = \text{MAXPooling}(v_1), \quad v_1^{\text{avg}} = \text{AvgPooling}(v_1)$$

$$v_2^{\max} = \text{MAXPooling}(v_2), \quad v_2^{\text{avg}} = \text{AvgPooling}(v_2)$$

最后将经过处理的向量拼接起来, 连接一个全连接层进行预测:

$$v = [v_1^{\max}; v_1^{\text{avg}}; v_2^{\max}; v_2^{\text{avg}}]$$

$$o = \text{Softmax}(Wv + b)$$

社区问答数据集

- ▶ MRPC 和 QQP 是最常用的评价语义匹配模型的基准数据集, LCQMC 是由哈尔滨工业大学构建的中文语义匹配数据集。

数据集	数据来源	训练集	测试集
MRPC	新闻抽取	3.7k	1.7k
QQP	社区问答抽取	367k	390k
LCQMC	真实对话抽取	239k	12k
PAWS-X	翻译构造	49k	2k

10.5 开放领域问答

开放领域问答

- ▶ 开放领域问答是与搜索引擎类似的形式，对于一个给定的询问 Q ，开放领域问答系统需要先从规模非常大的文档集合或知识库 (如 Wikipedia 或互联网) $D = \{d_1, \dots, d_n\}$ 中选择与问题最相关的文档 d ，然后通过文档 d 中包含的信息，给出询问 Q 的答案 A 。从任务定义中可以发现，开放问答的结果过程一般分为两个阶段：文档检索和阅读理解。



血压170严重吗



百度一下

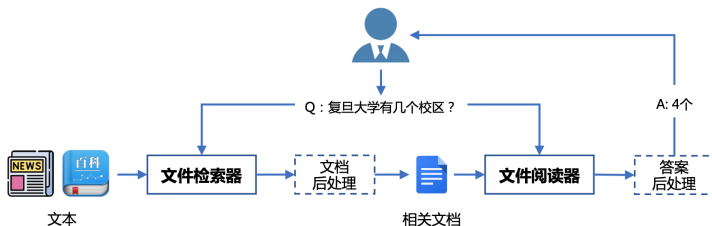
严重

“如果高血压达到170mmHg,是非常严重的。因为这已经属于高血压里最高级别的高血压,一旦造成脑血管的出血或者破裂,随时会有生命危险,所以一旦出现高血压170mmHg的情况要及时进行治疗。”

[高血压170严重吗-有来医生](#)

检索-阅读理解架构的开放问答模型

- 由于开放问答任务面对的是大量的文档或网页，为了兼顾准确率与性能，开放问答一般会采用文档检索方法快速筛选出与询问最相关的文档，在筛选出相关文档之后再通过阅读理解的方式给出问题的答案，这种架构被称为检索-阅读理解架构。



该架构的基本流程框架如图所示，主要包含文章检索器和文章阅读器两个模块，此外有些算法中还包含对检索得到的文档的后处理以及对抽取答案的后处理。

DrQA 开放领域问答算法

- ▶ DrQA 是利用维基百科作为知识库来实现开放式问答系统，DrQA 算法介绍对于任何事实性问题，系统都能从维基百科中找到一个包含答案的文章，在此基础上通过阅读理解算法从相关文章中提取答案。
- ▶ DrQA 利用 TF-IDF 的方法表示每个问题，并利用然后利用余弦相似度从大规模语料库中筛选出 5 篇最相关的文章。
- ▶ 答案抽取模块采用多层 RNN 网络，给定一个问题 $q = \{q_1, \dots, q_l\}$ 和包含 n 个段落的相关文章，每个段落由 m 个词组成 $p = \{p_1, \dots, p_m\}$ 。
- ▶ 对于段落中的每一个词 p_i ，DrQA 采用 Glove 词向量将其映射成一个 300 维的特征向量并把他们输入到 RNN 中并得到上下文向量 $H = \{h_1, \dots, h_m\}$:

$$H = \text{RNN}(p_1, \dots, p_m)$$

DrQA 采用 RNN 网络对问题 q 进行编码得到问题的上下文向量 $H^q = h_1^q, \dots, h_l^q$ 。

DrQA 开放领域问答算法

- 在预测时，DrQA 目标是在 p 中找到一个片段 $\{p_l, \dots, p_r\}$ 作为问题的答案。所以，DrQA 将编码好的段落表示向量 H 和问题表示向量 H^q 作为输入，分别训练两个二分类器预测答案的开始位置和结束位置：

$$P_{\text{start}}(i) = \exp(p_i W_s q)$$

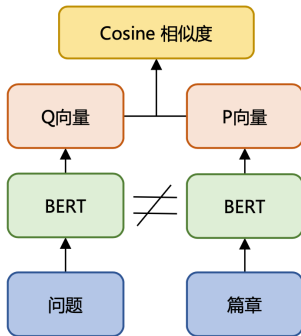
$$P_{\text{end}}(i) = \exp(p_i W_e q)$$

其中， W_s 和 W_e 是可训练的参数矩阵。

最后，DrQA 选择最佳的文章片段 $\{p_l, \dots, p_r\}$ 使得 $P_{\text{start}}(l) \times P_{\text{end}}(r)$ 最大。

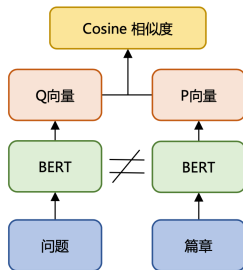
DPR 开放领域问答算法

- ▶ BM25、TF-IDF 等稀疏向量空间模型，通过词频和逆文档频率，对问题和文档利用关键词进行匹配，无法理解文档和问题中包含的语义信息。基于密集向量空间的 DPR (Dense Passage Retriever) 模型试图解决上述问题。
- ▶ DPR 模型主要在文章检索模块上做出了改进，DPR 采用了双编码器的结构，可以在 M 个文档中找到与问题 Q 最相关的 K 个文档。
- ▶ DPR 主要分为两个部分：编码和相似度度量模块。



DPR 开放领域问答算法

- ▶ 在编码部分，DPR 采用了双编码器结构，部署了两个独立的 BERT 模型对问题和文档分别进行编码，将两段文本分别映射到两个 d 维向量。
- ▶ DPR 采用点积的方式对两个向量的相似度进行度量。
- ▶ DPR 的编码模型采用一个正例文档和 n 个负例文档的方式进行训练。



在推理阶段，DPR 模型预先将所有文档进行编码并储存，利用 FAISS 向量比较工具²对相应的问题进行检索，最终找到与问题最相关的前 K 个文档。

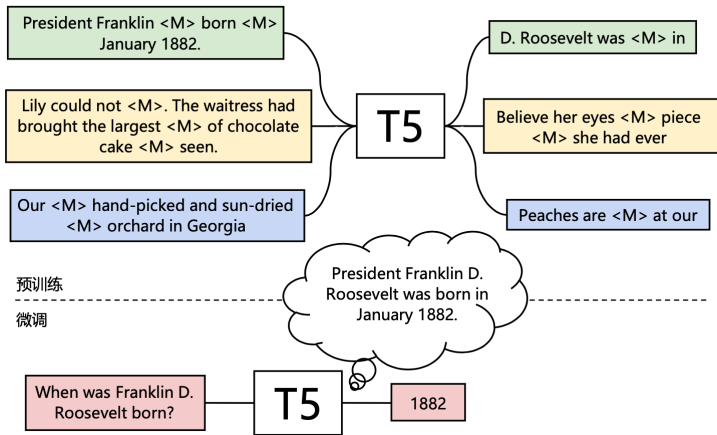
²Faiss 是一个由 facebook 开发以用于高效相似性搜索和密集向量聚类的库。

端到端架构的开放问答模型

- ▶ 随着预训练语言模型的规模越来越大，训练数据越来越多，一些研究人员认为在这些在大规模数据上训练的超大规模语言模型很可能已经建模了网页数据中的知识。基于端到端架构的开放问答模型引起了广泛兴趣。
- ▶ **PLSLM**采用小样本提示学习的方式，利用网络数据对超大规模语言模型进行增强，提示学习的方法不需要重新进行预训练或者改变模型结构，可以轻量级的实现模型信息更新。基于小样本提示学习增强大规模语言模型的方法分为三步：
 - 1) 根据用户提出的问题 q ，采用搜索引擎检索一系列相关文档作为增强信息。
 - 2) 利用检索出的文档，利用提示学习对模型进行调节。
 - 3) 模型通过每一个检索出的文档生成候选答案，并对这些候选答案进行重排序并选出最合适的作为最终答案返回。

端到端架构的开放问答模型

► 端到端架构开放问答模型 (以 T5 为例)



端到端架构的开放问答模型

- ▶ 对于给定的问题 q , PLSLM 算法利用搜索引擎检索得到前 20 个最相关文本作为模型的增强文档。
- ▶ 由于网络上获取的文档会伴随噪音等问题，会增强后续提示学习的难度，所以 PLSLM 对检索文档进行了预处理。将所有检索到的文档分为多个段落，其中每个段落由 6 个句子表示，最后再利用 TF-IDF 算法对所有段落和 q 进行编码，利用余弦相似度进行重排序，将这些排序后的段落作为增强数据。
- ▶ 得到经过排序的段落 P 之后，PLSLM 利用小样本提示学习对大规模预训练模型进行调节，PLSLM 针对开放领域问题回答设计了相应的提示模板。

端到端架构的开放问答模型

- ▶ **提示学习**会通过提示模板引导模型根据问题和证据去生成合适的答案。在对模型进行增强之后，PLSLM 利用打分函数对所生成的候选答案在此排序，挑选其中最合适的作为最终答案返回给用户。针对问题 q 的候选答案 a_i 以及 n 个检索片段 p_i 可以采用如下三种方法选择最合适的答案：

(1) 直接推理: 最大化 $P(a|q)$

$$P(a|q) = \sum_{i=1}^n P_{\text{TFIDF}}(p_i|q) \cdot P(a_i|q, p_i)$$

(2) 噪声信道模型: 最大化 $P(a_i, q|p_i)$

$$P(a_i, q|p_i) = \frac{P(q|a_i, p_i) \cdot P(a_i|p_i)}{P(q|p_i)}$$

(3) 专家乘积 (Product of Experts, PoE): 使用额外的语料训练权重综合上述概率

端到端架构的开放问答模型

- ▶ 通过提示学习利用大规模语言模型直接进行开放问答的方式成为近年来开放问答任务上的一个新范式。
- ▶ 随着规模更大，预训练更充分的生成式模型 (如 GPT-3) 不断涌现，这种端到端架构的开放问答模型也逐渐成为研究热点。

开放领域问答语料库

- ▶ SQuAD_{open} 扩展自 SQuAD 数据集，在应用于开放问答时，使用整个维基百科数据集合作为问答资源，不再提供问题所对应的文章段落。由于 SQuAD 评测不公开测试集合，因此在应用于开放问答场景评测时，使用开发集用于测试。与 SQuAD_{open} 扩展自 SQuAD 类似，包括 MS-MARCO、HotpotQA 等在内的用于阅读理解评测的语料集合常通过不提供答案所在文章，从而应用于开放领域问答评测。

数据集	知识来源	数据集规模
SQuAD _{open}	维基百科	97K
Natural Questions	维基百科	323K
XQA	维基百科	90K
SearchQA	搜索引擎	140K

本章小结

第十章 智能问答

10.1 智能问答概述

10.2 阅读理解

10.3 表格问答

10.4 社区问答

10.5 开放领域问答